



VERSION 1 · BUILD REPORT

What was built, how it works & what's next.

AI fitness web app: FastAPI + PostgreSQL + Gemini · React + GSAP + three.js

Prepared for Nibir Borkakati
3 October 2026

SECTION 01

Summary

FitAI v1 is now a working web app from sign-up to coaching. A user creates an account, answers **13 questions one at a time** with animated transitions, and lands on a dashboard. The dashboard shows **11 health metrics computed in Python** without calling Gemini, a rule-based weekly training split, a **Gemini-generated workout plan** and an **AI coach chat** that knows the user's profile, metrics and plan.

Area	Status
Authentication	Register, login, JWT session, logout, protected routes, auto-redirects
Onboarding	13 single-question steps (2 optional), keyboard shortcuts, saved draft, review and edit screen
Python calculations	BMI, healthy weight range, BMR, TDEE, calorie target, macros, water, HR zones, goal timeline, weekly split, sets/reps/rest
Gemini	Structured-JSON weekly plan, contextual coach chat, retry + fallback model, daily limit
UI	Dark theme with lime and violet accents, Sora + Inter fonts, GSAP + Lenis smooth scrolling, curtain page transitions, animated 3D robot
Quality	19 automated backend tests passing, production build clean, full browser walkthrough done

Action needed from you: your database password (*Admin@123*) was shared in our chat and is a weak password. It is only stored in *backend/.env*, which is git-ignored, but please change it before deploying anywhere, and use a long random *SECRET_KEY* in production.

SECTION 02

What I found in the existing code

The backend foundation (FastAPI, SQLAlchemy 2, Alembic, JWT with Argon2) was solid. These are the issues I found and what I did about each one:

Finding	Impact	Fix
<i>requirements.txt</i> was saved as UTF-16 (from PowerShell's <i>pip freeze ></i>)	<i>pip install -r</i> fails on most setups	Rewritten as UTF-8, pytest added
<i>gemini-2.5-flash</i> is no longer available to this API key	Every AI call would return 404	Switched to <i>gemini-3.8-flash</i> , with <i>gemini-flash-lite-latest</i> as fallback
<i>gemini_service.py</i> was empty	No AI features	Full service written (section 5)
Goal, activity level etc. were free-text strings	An unknown activity level crashed TDEE with a 500 error	Strict <i>Literal</i> validation on every choice field (now a 422 error)
Profile could only be created (POST returned 409 the second time)	Users could not edit their answers	Added <i>PUT /api/profile</i> to create or update
BMR treated any non-male user as female	Wrong numbers for 'other'	'other' uses the midpoint of both equations
No CORS middleware	A deployed frontend could not call the API	Configurable <i>CORS_ORIGINS</i>
No workout-preference field; sleep, diet and target weight were required	Didn't match the v1 question list	New column added; those three made optional
Alembic <i>env.py</i> imported only <i>User</i>	New models would be missed by autogenerate	Imports the whole models package
<i>datetime.utcnow</i> (deprecated in Python 3.12+)	Deprecation warnings	Shared <i>utcnow()</i> helper
Emails were case-sensitive	<i>A@x.com</i> and <i>a@x.com</i> counted as two accounts	Emails are lower-cased

Note: migration *9c8fb285b8d0* is an empty duplicate of 'create users table'. It is harmless, so I left it in place to keep your migration history intact.

SECTION 03

Architecture

Request flow: React (Vite dev server on :5173) proxies */api* to FastAPI on :8000. FastAPI reads and writes Postgres and only calls Gemini on two endpoints: plan generation and chat.

Layer	Technology
Frontend	React 19, TypeScript, Vite 8, React Router 7
Animation	GSAP 3.15 (ScrollTrigger, SplitText, @gsap/react), Lenis smooth scroll
3D	three.js, @react-three/fiber, @react-three/drei; 'RobotExpressive' model (CC0)
Fonts / icons	Sora (display), Inter (body), self-hosted via Fontsource; lucide-react icons
Backend	FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, PyJWT, pwdlib (Argon2)
Database	PostgreSQL (fitai_db): users, profiles, workout_plans, chat_messages
AI	google-genai SDK, structured output (response_schema), low thinking level

New and changed backend files

File	Purpose
app/services/calculations.py	All metric formulas plus the rule-based training template (extended)
app/services/gemini_service.py	Prompt building, retries and fallback, plan and chat generation (new)
app/services/profile_service.py	Profile lookup and report builder (new)
app/services/ai_usage.py	Per-user daily AI limit (new)
app/routers/metrics.py, plan.py, coach.py	New endpoints
app/routers/profile.py, auth.py, health.py	Upsert endpoint, has_profile on /me, calorie floor
app/models/workout_plan.py, chat_message.py	New tables (cascade-delete with the user)
app/schemas/profile.py, health.py, ai.py	Validated enums, report schema, AI response schemas
alembic/versions/a6129dc02caa_...py	Migration (applied): new tables and profile columns
tests/ (3 files)	19 tests: calculations plus the API with a fake Gemini

Frontend structure

Path	Purpose
src/lib/api.ts, auth.tsx, motion.tsx, types.ts	API client, auth context, Lenis + GSAP setup, shared types
src/components/PageTransition.tsx	Two-panel curtain transition on every route change (including back/forward)
src/components/Robot.tsx, LazyRobot.tsx	3D mascot: recoloured, follows the cursor, gestures; lazy-loaded

Path	Purpose
src/components/Navbar, RouteGuards, CountUp	Navigation, protected and guest-only routes, animated numbers
src/pages/Landing, Auth, Onboarding, Dashboard, Coach	The five screens (each has its own CSS file)
src/pages/onboarding/questions.ts	The question flow defined as data: easy to add or reorder questions

SECTION 04

Python calculations (no AI involved)

`GET /api/metrics` returns everything below in a few milliseconds without calling Gemini. The same numbers are also passed to Gemini as context, so the AI never has to work them out itself.

Metric	Method
BMI and category	kg / m ² ; WHO bands (18.5 / 25 / 30)
Healthy weight range	Weight at BMI 18.5 and 24.9 for the user's height
BMR	Mifflin-St Jeor (male +5, female −161, other −78 midpoint)
TDEE	BMR × activity factor (1.2 / 1.375 / 1.55 / 1.725 / 1.9)
Calorie target	Fat loss: −20% (max −500 kcal), never below BMR or 1200/1350/1500. Muscle gain: +10% (max +300)
Macros	Protein 1.6–2.0 g/kg by goal (BMI-25 reference weight when BMI is high); fat 25% of kcal; carbs the remainder
Hydration	35 ml/kg on rest days, +0.5 L on training days
Heart-rate zones	Tanaka max HR = 208 − 0.7 × age; five zones from 50% to 100%
Goal timeline	0.5 kg/week loss or 0.25 kg/week gain to the target weight
Weekly split	Day pattern by days per week (e.g. 4 = Mon/Tue/Thu/Fri); split by preference and level (Full Body, Upper/Lower, PPL, HIIT hybrid, mobility...)
Prescription	Sets/reps/rest/cardio by goal; set count adjusted for fitness level; intensity note (RIR)

Example (30 y male, 180 cm, 85 kg, fat loss, moderately active): BMI 26.2, BMR 1,830, TDEE 2,837, target 2,336 kcal (−500), 162 g protein / 276 g carbs / 65 g fat, max HR 187, 14 weeks to 78 kg.

SECTION 05

Gemini integration and keeping API load low

- **Python does the maths.** Gemini receives a roughly 10-line summary (profile, metrics, the weekly schedule and the default prescription) and only has to choose exercises and write coaching text.
- **Structured output.** Plans use *response_schema* (Pydantic), so the JSON always matches. No re-asking and no fragile parsing. The days are re-ordered to match the Python schedule.
- **Plans are saved.** Opening the dashboard reads the plan from Postgres. Gemini runs only when the user presses Generate or Regenerate, and the UI flags when the profile has changed since the plan was made.
- **Cheap settings.** *thinking_level='low'*, capped output tokens, automatic function calling turned off.
- **Short chat memory.** Only the last 6 messages go to Gemini; the system prompt holds the profile context.
- **Resilience.** On 429/5xx errors it retries once with a delay, then falls back to *gemini-flash-lite-latest*. If both fail, the user sees a friendly 503 message. Failed calls are not saved and don't use up quota.

- **Daily limit.** *AI_DAILY_LIMIT* (default 40) per user, shown as a meter on the coach page.
- **Safety.** The prompts require injury-aware substitutions, only the listed equipment and no diagnoses. The coach declines off-topic questions.

Measured during testing: a plan took about 10 s on *gemini-3.8-flash*. In one run the primary model returned 'high demand' errors, and the fallback produced the plan in about 25 s. A coach reply takes a few seconds.

SECTION 06

API endpoints

Method	Path	AI call?	Description
POST	/api/auth/register · /login	No	Returns a JWT (24 h)
GET	/api/auth/me	No	id, email, has_profile
GET / PUT	/api/profile	No	Read or create/update the questionnaire answers
GET	/api/metrics	No	Full Python health report
GET	/api/plan	No	Latest saved plan, or null
POST	/api/plan/generate	Yes	New plan; saved with a metrics snapshot and the model name
POST	/api/coach/chat	Yes	Reply plus remaining_today
GET / DELETE	/api/coach/history	No	Last 50 messages / clear
GET	/api/coach/usage	No	AI calls used and remaining today
POST	/api/health/calculate	No	Public calculator (no login)

Interactive docs are available at <http://127.0.0.1:8000/docs> while the backend is running.

SECTION 07

UI and UX design

Colour system

	Ink #07090d Background		Lime #c8f65d Primary actions, energy
	Violet #8b7cff AI / secondary accent		Coral #ff7a59 Warnings, deficits
	Sky #4cc9f0 Hydration		Amber #ffc857 Fat macro, caution

Typography: **Sora** (geometric display face for headings and numbers) and **Inter** (body text), both self-hosted. Glass-style cards, subtle grain and soft background glows.

Motion

- **Smooth scrolling:** Lenis driven by GSAP's ticker, kept in sync with ScrollTrigger.
- **Page transitions:** a lime panel and a dark panel wipe up, the route changes underneath, and they wipe away. Uses an intro logo reveal on first load. Works with links, redirects and the browser's back/forward buttons.
- **Landing:** SplitText headline reveal, parallax stat cards, a marquee that reacts to scroll speed, a pinned 'How it works' section with scroll-scrubbed steps, batch reveals and count-up stats.

- **Onboarding:** one question per screen. Answers slide and blur out, and the next slides in, direction-aware. Choice cards auto-advance, there are sliders with +/- buttons, and keyboard shortcuts (1–9, Enter, Esc).
- **Dashboard:** numbers count up, the BMI marker slides into place, the macro donut draws itself, HR bars grow, the water glass fills, and day tabs animate.
- **3D robot:** follows the cursor and reacts: waves on arrival, runs during plan generation, jumps when the plan is ready, shakes its head on errors, gives a thumbs-up at the review step.
- Respects *prefers-reduced-motion*, and the layout is responsive down to 390 px with no horizontal scroll.

SECTION 08

Onboarding questions

#	Question	Input	Used for
1	Gender	Cards	BMR
2	Age	Slider + number	BMR, HR zones
3	Height (shows ft/in)	Slider + number	BMI, BMR
4	Weight (shows lb, live BMI)	Slider + number	Everything
5	Main goal	Cards (5)	Calories, protein, reps
6	Target weight (optional, hidden for Maintain)	Slider	Timeline
7	Fitness level	Cards (3)	Sets, split, intensity
8	Daily activity	Cards (5)	TDEE
9	Days per week	1–7	Schedule
10	Session length	20–90 min	Plan duration
11	Workout preference	Cards (5)	Split type
12	Equipment	Cards (5)	Exercise choice
13	Injuries / limitations (optional)	Chips + text	Safety in prompts

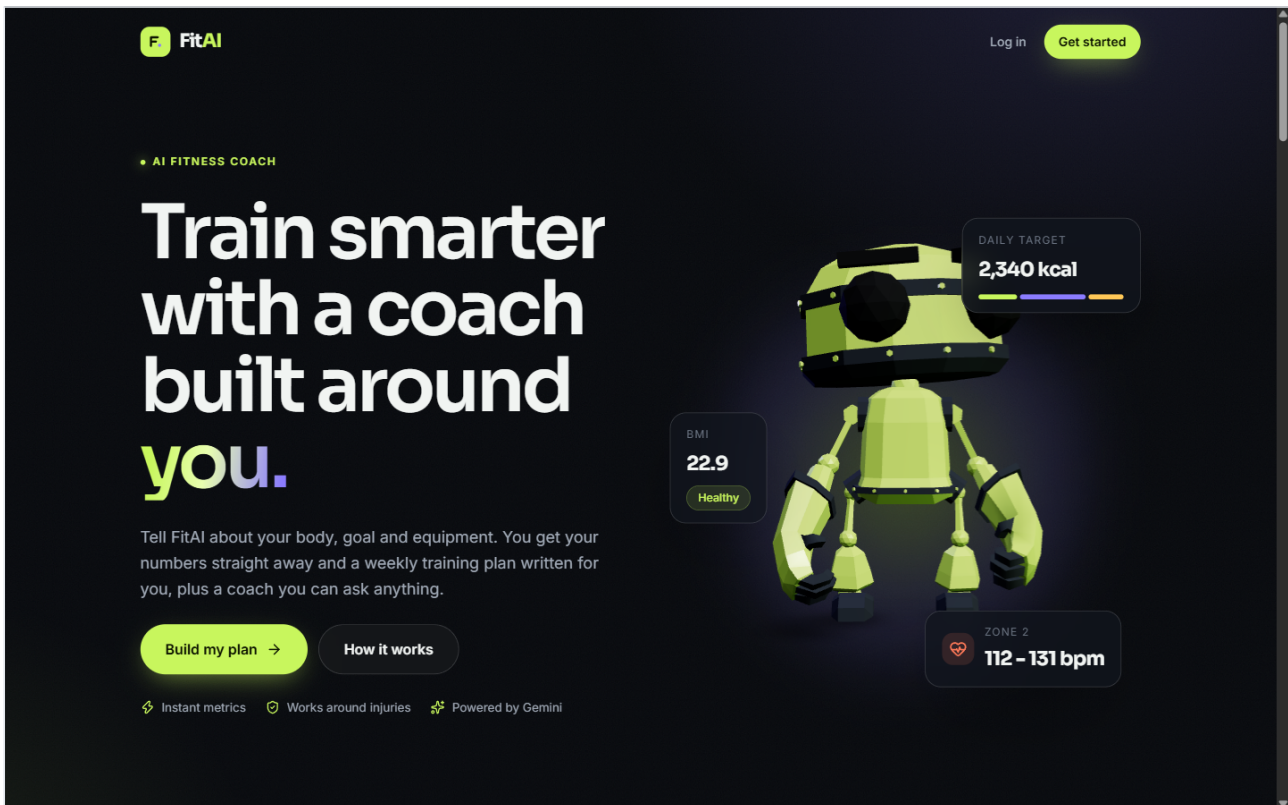
SECTION 09

Testing and verification

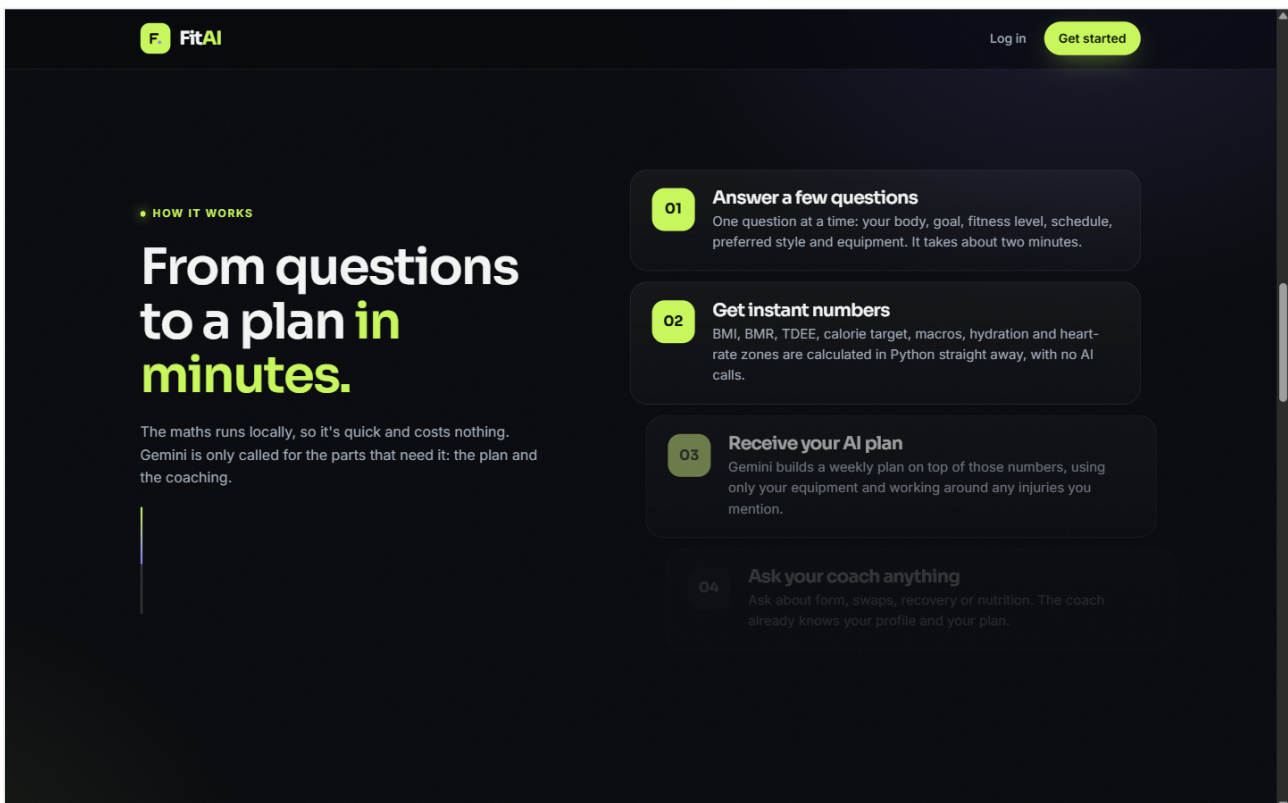
- **Backend: 19/19 pytest tests pass.** Formulas (BMI, BMR, TDEE, calories with floors, macros, water, HR, timeline, split), register/login/me, profile upsert and validation, metrics, plan save and ordering, chat history, the daily limit (429) and AI outage (503, nothing saved). The tests use in-memory SQLite and a fake Gemini.
- **Frontend:** TypeScript strict build passes; oxlint shows 0 errors (only React-Compiler style warnings).
- **End-to-end in a real browser (Playwright):** landing, sign-up, all 13 questions, review, dashboard, real Gemini plan, day tabs, real coach chat, logout, and a mobile (390 px) layout check.
- Bugs found and fixed during the walkthrough: the curtain sticking after a CSS/GSAP transform conflict, the gradient word hidden by SplitText, metric grid gaps, onboarding lock-up after a hot reload, and the dashboard spinning forever on API errors (it now shows a retry option).
- The demo account used for testing (fitai.demo@example.com) and its data were deleted afterwards. Your existing user (test@fitai.com) was not touched.

SECTION 10

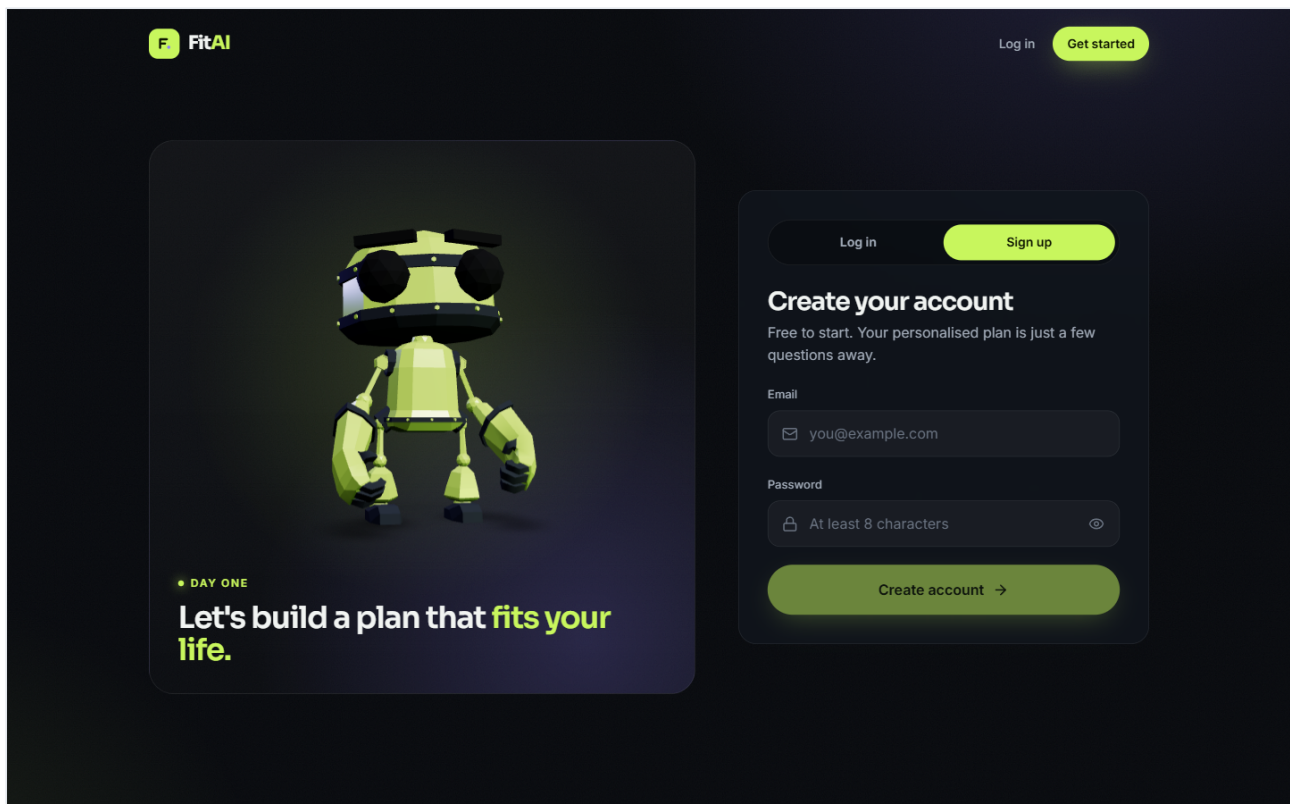
Screens



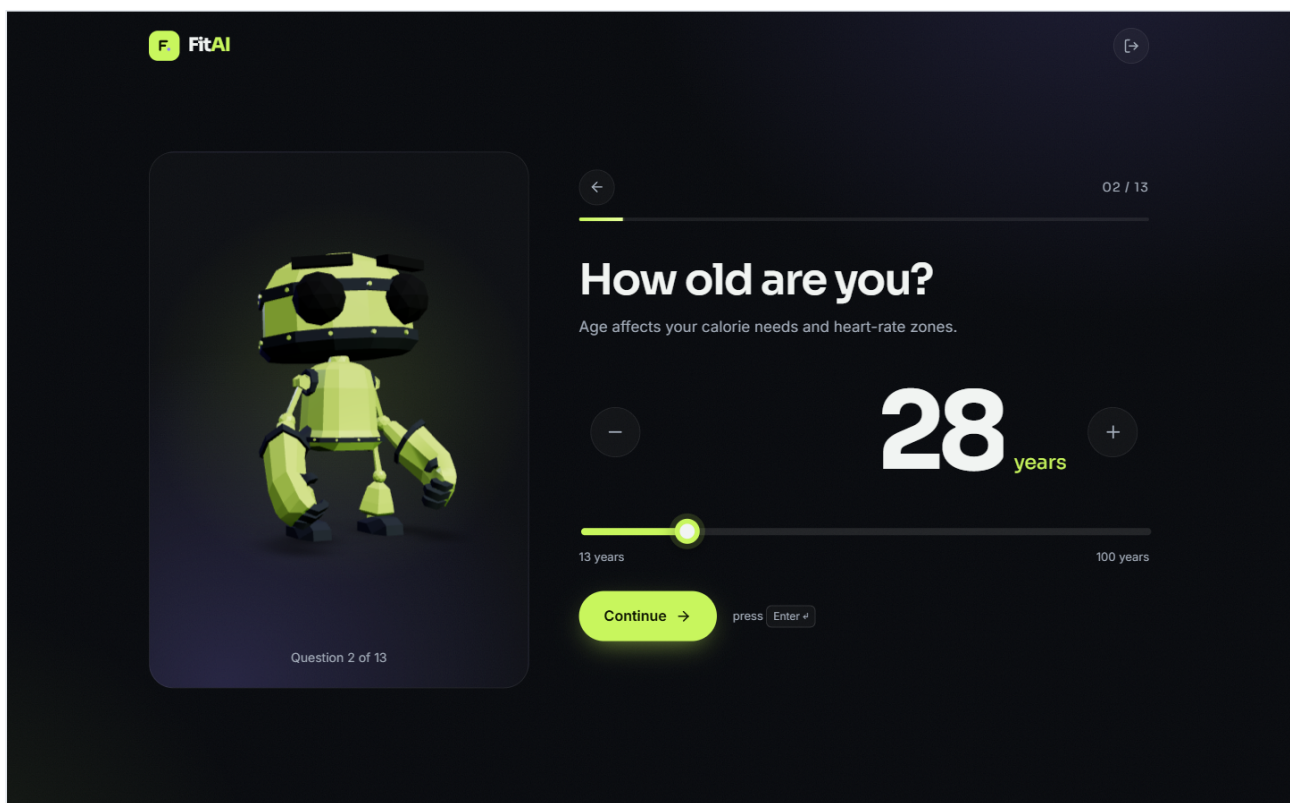
Landing hero: SplitText headline, 3D robot, parallax stat cards.



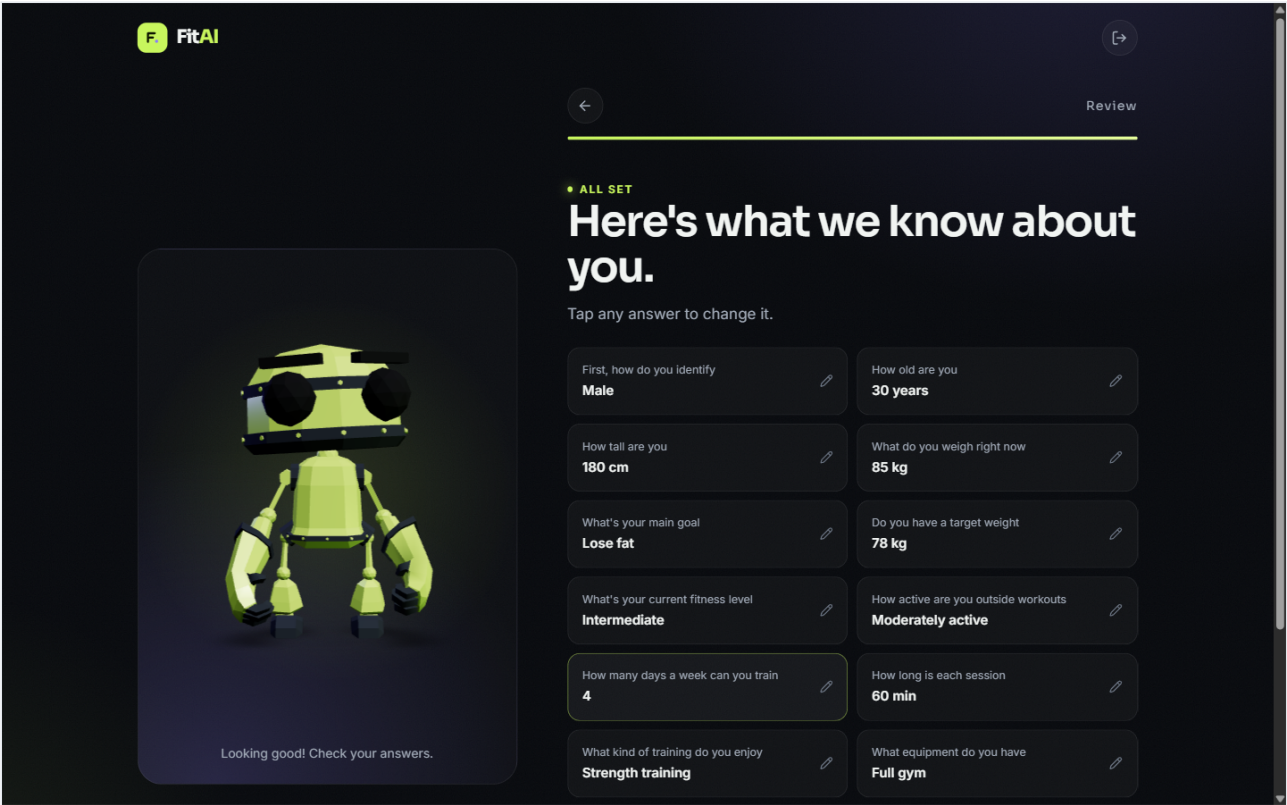
Pinned 'How it works' section; steps light up as you scroll.



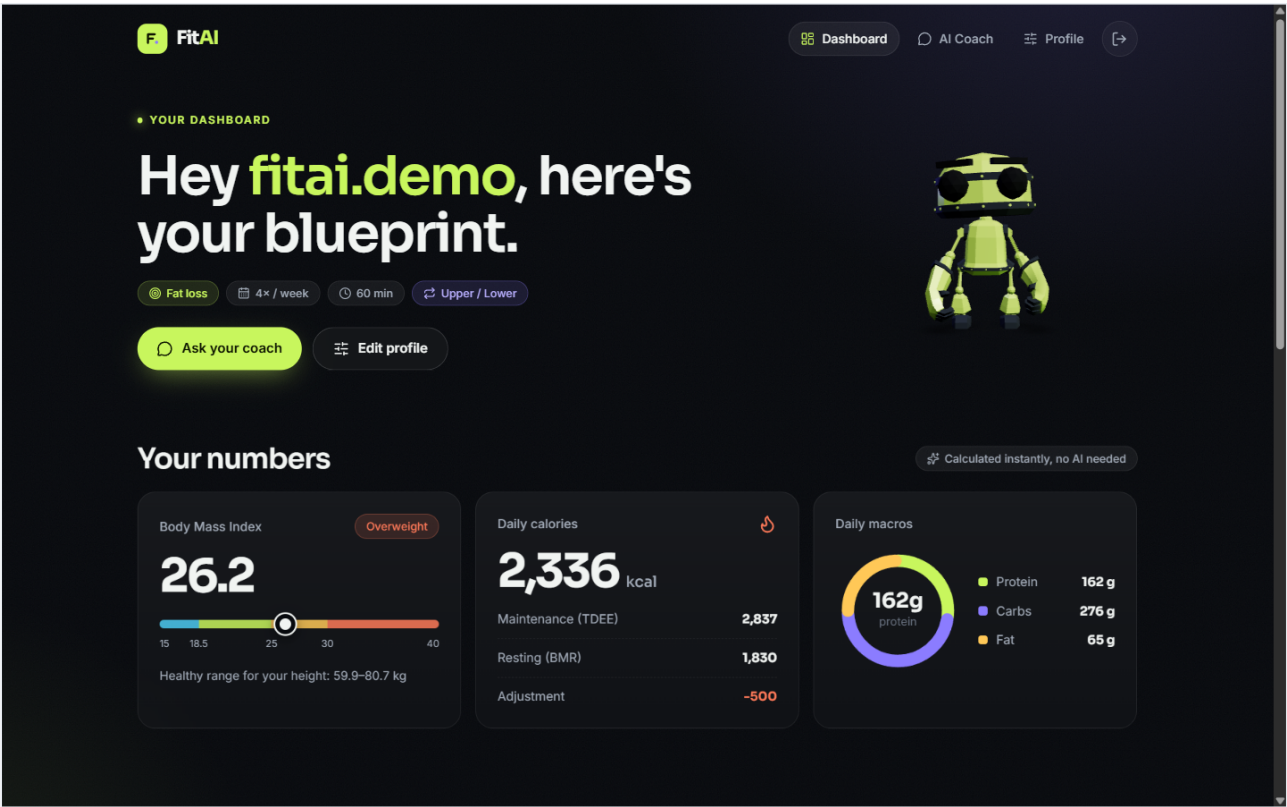
Sign-up and login with a sliding tab switch and password strength meter.



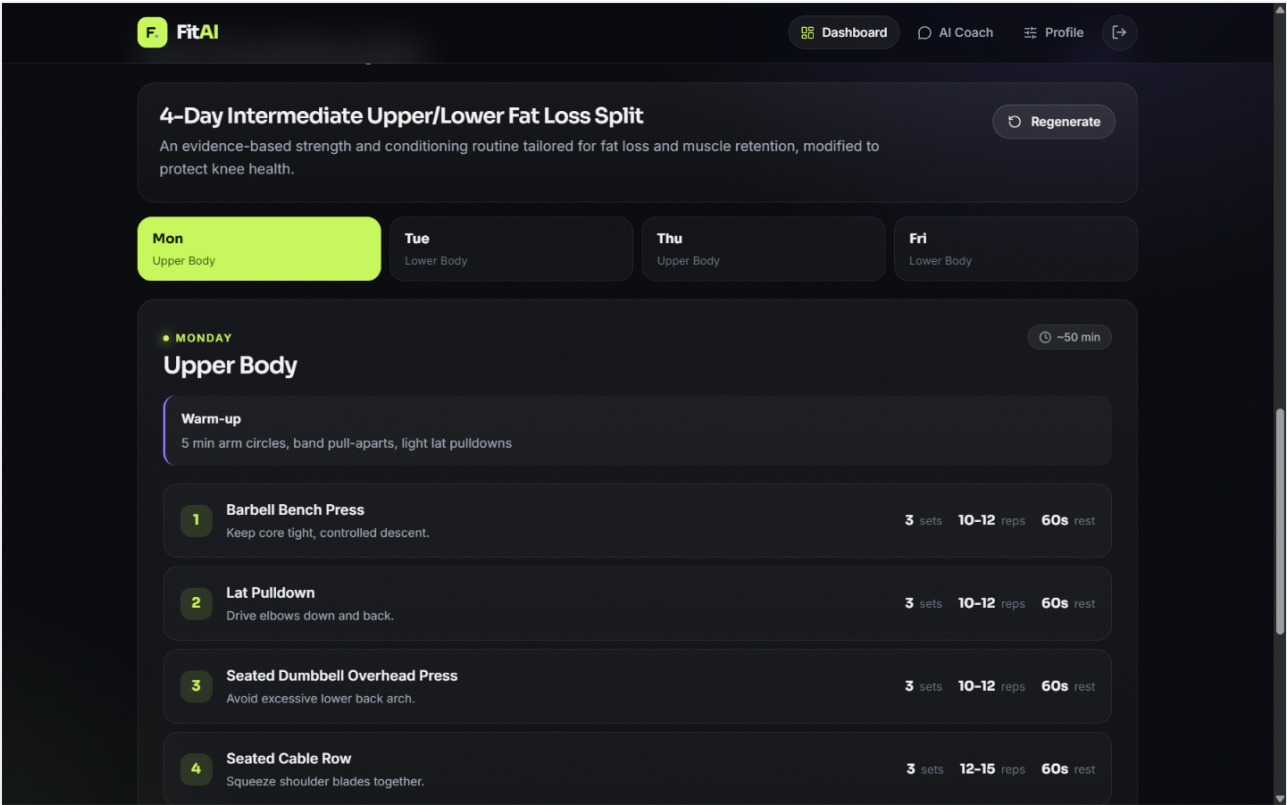
Onboarding: one question at a time, with a progress bar.



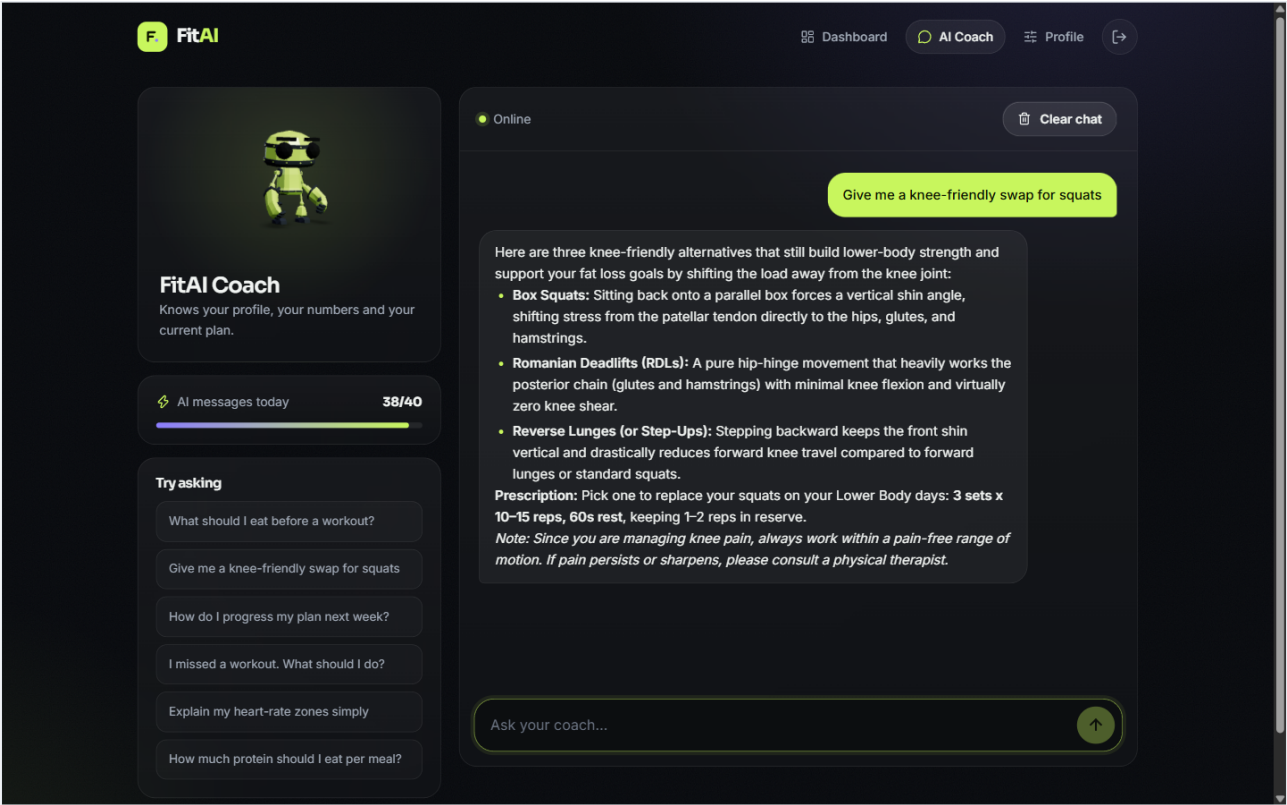
Review screen: tap any answer to jump back and edit it.



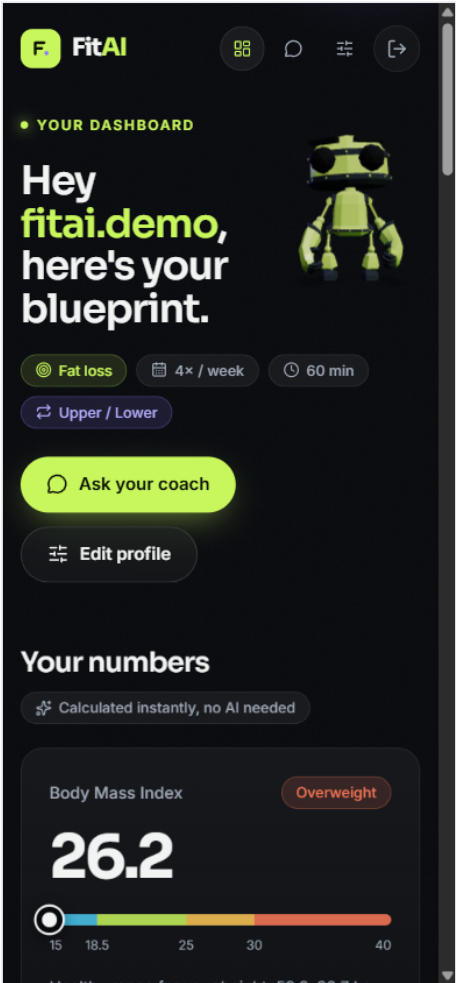
Dashboard header and Python-computed metrics.



Gemini plan: day tabs, warm-up, exercises with sets × reps × rest.



AI coach answering with knowledge of the user's knee injury and prescription.



Mobile layout (390 px).

SECTION 11

How to run

Backend (PowerShell):

```
cd backend
.venv\Scripts\activate
alembic upgrade head (already applied)
uvicorn app.main:app --reload --port 8000
```

Frontend:

```
cd frontend
npm install
npm run dev then open http://localhost:5173
```

Tests: *cd backend* then *.venv\Scripts\python -m pytest tests -q*. Full details are in README.md.

Known limitations in v1

- The JWT is stored in localStorage and there are no refresh tokens, password reset or email verification yet.
- Plan generation is a blocking request (about 10–25 s) with no streaming.
- The 3D chunk is about 990 kB (265 kB gzipped). It is lazy-loaded, but a compressed (Draco) model would make it lighter.
- @react-three/fiber logs a harmless 'THREE.Clock deprecated' console warning (it comes from the library).
- On Windows, uvicorn --reload occasionally failed to restart; if the API looks out of date, restart it.
- There's no rate limiting on login yet, and no workout logging or progress history.

SECTION 12

Suggestions for version 2

Grouped into three phases, in the order I'd tackle them. Phase 1 makes users come back, phase 2 makes the AI smarter and cheaper, and phase 3 gets the app ready for real users.

Phase 1: Retention (track and adapt)

Feature	Why	How
Workout logger and 'player mode'	Turns the plan into something users do every day	Per-set logging (weight, reps, RPE), rest timer, the robot demonstrating; new workout_logs table
Progressive overload engine	Plans get better automatically	Python rules: hit the top of the rep range for every set, then +2.5–5% next session. No AI needed
Weekly check-in	Keeps calories accurate as weight changes	Weight, energy, sleep; Python recalculates TDEE and the target, and suggests replanning only when needed
Progress charts	Motivation	Weight trend (7-day average), volume per muscle, adherence streaks
Nutrition basics	Diet matters as much as training	Macros per meal, simple food log, dietary preference (column already exists)
Reminders and calendar	Habit building	Email/push reminders, .ics export of the training days

Phase 2: Smarter and cheaper AI

Idea	Benefit
Curated exercise library (Postgres) with equipment, muscle and contraindication tags; Gemini picks IDs	Shorter outputs, no made-up exercises, demo GIFs/videos, easy swaps without any AI call
Plan cache keyed by a hash of the inputs (goal, level, days, equipment, preference, injuries)	Identical profiles reuse a plan, so many plan generations cost nothing
Streaming responses (SSE) for chat and plans	Text appears within about a second, which makes the coach feel much faster
Background jobs (ARQ/Celery) for plan generation	No long HTTP requests; retries happen off the request path
Gemini context caching and token-usage logging per call	Lower cost, plus visibility into spend per user and feature
Small knowledge base (RAG) for the coach	Answers grounded in your own vetted content
Coach tools: 'swap exercise', 'move session'	The chat can actually edit the plan, not just give advice

Phase 3: Production readiness

- **Auth:** httpOnly refresh-token cookies, email verification, password reset, Google sign-in, login rate limiting (slowapi).

- **Safety:** a PAR-Q style health screening question set; flag high-risk answers and recommend seeing a doctor.
- **Integrations:** Google Fit / Apple Health / Strava for steps, heart rate and workouts, with automatic activity level.
- **Platform:** PWA (installable, offline plan view), later a React Native/Expo app reusing the same API.
- **DevOps:** Docker Compose, GitHub Actions (pytest, build, Playwright E2E), managed Postgres, Sentry, secrets manager.
- **Experience:** light theme toggle, i18n (e.g. Hindi/Assamese), accessibility audit, gamification (streaks, badges).

Recommended first steps for v2: (1) workout logger with automatic progression, (2) exercise library with ID-based plans and caching, (3) streaming chat. Together they give the largest gains in user value and in AI cost reduction for the effort.